ENTERPRISE

Go-Live Checklist & Setup Guide

Exact, step-by-step instructions to take HeyVacay Enterprise from staging to production — every integration, where to go, and precisely what to click. Items marked **DONE** are already built and deployed; items marked **ACTION NEEDED** are yours.

Prepared for Danil B. · ent.stage.heyvacay.co · AWS account 256309399831 · us-west-2

DONE built & shipped by us ACTION NEEDED a key, DNS record, or merge you perform

0 How deploys work DONE

So the steps below make sense, here is the pipeline that is already wired up.

Staging app	https://ent.stage.heyvacay.co (live now)
Deploy trigger	Merging a PR into the `stage` branch auto-deploys. (Or run the **Stage Web Enterprise - Deploy** workflow manually from the Actions tab.)
AWS runtime	ECS Fargate — cluster `stage-heyvacay-cluster`, service & task `stage-heyvacay-ent`, container name `ent`, region `us-west-2`.
Infra as code	The task definition (CPU, memory, env vars, secrets) lives in the **heyvacay-iac** repo → `ent.tf`.
Current branch	All the work described here is on `claude/real-search`, ready to merge.

Every integration below follows the **same pattern**: the app already calls the live API when its secret is present, and falls back to realistic demo data when it isn't. So "turning it on" always means the same three moves — **(a)** store the key in AWS Secrets Manager, **(b)** reference it in `ent.tf`, **(c)** redeploy. No code changes needed.

1 Ship everything to production ACTION NEEDED

Promote the completed work so it is live on the real domain.

1. **Open the pull request.**
   - Go to `github.com/heyvacay/heyvacay-ent` → **Pull requests** → **New pull request**.
   - Base = `stage`, compare = `claude/real-search`. Click **Create pull request**, then **Merge**.
   - The merge auto-runs the deploy workflow. Watch it under the **Actions** tab → **Stage Web Enterprise - Deploy**.
2. **Verify staging.** When the workflow is green, open https://ent.stage.heyvacay.co and spot-check sign-in, a booking, and Nova.
3. **Promote to production** (once a prod environment/branch exists — e.g. `main` → `ent.heyvacay.co`). Repeat the merge into the production branch. If production infra isn't stood up yet, see step 8 (DNS).

2 Payments — Stripe (same processor as heyvacay.co) REFERENCE & REDEPLOY

The checkout already initializes payment through the shared HeyVacay backend and the same tracking tags. The Stripe keys are **already in AWS Secrets Manager** — all that's left is to reference them on the Enterprise container and redeploy.

Where: heyvacay-iac → then redeploy. (Keys already in Secrets Manager — nothing to create.)

1. **Keys are already stored.** The same live **Secret key** (`sk_live_…`) and **Publishable key** (`pk_live_…`) that heyvacay.co runs on are already in AWS Secrets Manager. Same Stripe account = same processor, same payout, same reporting — nothing to copy from the Stripe Dashboard. Just note the secret's ARN + JSON keys so you can reference them below.
2. **Set the publishable key (required).** Point the build-time env var `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` at the stored `pk_live_…` — the **same** value heyvacay.co uses. This is the one that makes checkout go live: with it present, the app mounts Stripe card entry and the **backend** (same account/keys/tags as heyvacay.co) creates the charge — exactly how RateHawk inventory works. A publishable key is not secret, so it can be injected as a plain env var at build time. `STRIPE_SECRET_KEY` is **optional** (only a direct-to-Stripe fallback) and is already available in Secrets Manager if you want to wire it.
3. **Reference them on the container.** In **heyvacay-iac** → `ent.tf`, add to the task definition's `secrets` block (one entry per key), pointing `valueFrom` at the existing secret ARN + JSON key. Run `terraform apply`.
4. **Confirm the payment-tracking endpoint.** The app POSTs to `{ENT_API_BASE_URL}/bookings/payment/init-stripe` with metadata tags `source: heyvacay-enterprise`, `product: corporate-travel`. Make sure `ENT_API_BASE_URL` points at the same backend heyvacay.co uses so charges land in the same ledger. (Defaults to `https://api.stage.heyvacay.co`.)
5. **Redeploy & test** (step 1). Do a $1 test booking; confirm it appears in the Stripe Dashboard → **Payments** with the metadata tags, then refund it.

> Already done for you: the full Stripe Elements checkout, saved-card option, the booking + **booking-confirmed** screens matched to heyvacay.co (including the confetti), and the metadata tags — plus the keys are already in Secrets Manager. Until they're referenced on the container, checkout stays in clearly-labelled DEMO MODE — no real charge.

3 Dining — OpenTable API ACTION NEEDED

Powers the restaurant directory, availability and reservations used by the Events dinner flow.

Where: OpenTable for Partners → AWS Secrets Manager → heyvacay-iac → redeploy.

1. **Request API access.** Go to https://platform.opentable.com (OpenTable Partner / Affiliate program). Sign in with the HeyVacay OpenTable partner account, or click **Become a partner** and complete the application for API access.
2. **Create an app / credentials.** In the partner portal → **Apps** (or **Credentials**) → **Create app**. OpenTable will issue either a single **API key** or an OAuth **Client ID + Client Secret**.
3. **Note the base URL.** Default is `https://platform.opentable.com`. If your partner docs specify a different host, keep it handy for `OPENTABLE_API_BASE`.
4. **Store the secret(s)** in Secrets Manager (as in step 2), using whichever you were issued:
   - `OPENTABLE_API_KEY`, **or**
   - `OPENTABLE_CLIENT_ID` + `OPENTABLE_CLIENT_SECRET`
   - (optional) `OPENTABLE_API_BASE` if non-default
5. **Reference in `ent.tf`** → `terraform apply` → **redeploy** (step 1).
6. **Verify.** Open the app → **Events → dinner** flow; live restaurants and times should now appear instead of demo data.

Already done: the full dining UI, search, RSVP and dinner-invite emails. The moment the key is present the same screens call OpenTable live.

4 Transactional email ACTION NEEDED

Sends approval requests, offers, event-added notices, dinner invites, and the C-Suite commission alerts.

Where: your email provider → Secrets Manager → heyvacay-iac → redeploy.

1. **Pick/confirm the provider heyvacay.co uses** (e.g. SendGrid, Postmark, or AWS SES). Log in and create an **API key** with send permission.
2. **Verify the sender domain** in the provider so mail from `notifications@heyvacay.co` passes SPF/DKIM. (For SES: Verified identities → add domain → publish the DNS records it gives you.)
3. **Store secrets:** `EMAIL_API_KEY` = the key; `EMAIL_FROM` = `notifications@heyvacay.co` (or your preferred from-address).
4. **Reference in `ent.tf`** → `terraform apply` → **redeploy**.
5. **Verify.** Submit a trip for approval; the approver should receive a branded email. Also confirm a C-Suite commission-limit alert sends when a rep exceeds the ±band.

5 Live trip data — Weather & Flight status ACTION NEEDED

Optional polish for the active-trip view. Everything works without them (demo data); add keys to go live.

Weather (OpenWeather)

1. Create a free key at https://openweathermap.org/api → **Sign up** → **API keys**.
2. Store `WEATHER_API_KEY` in Secrets Manager (base defaults to OpenWeather; override with `WEATHER_API_BASE` if needed).
3. Reference in `ent.tf` → apply → redeploy.

Flight status

1. Choose a provider (e.g. AviationStack aviationstack.com, or FlightAware AeroAPI). Sign up and copy the API key + its base URL.
2. Store both: `FLIGHT_STATUS_API_KEY` and `FLIGHT_STATUS_API_BASE` (both are required for it to go live).
3. Reference in `ent.tf` → apply → redeploy.

6 DNS, login screens & domain self-join ACTION NEEDED

The login screens are built and waiting; they go live the moment DNS points at the app.

Where: your DNS provider (Route 53 / registrar) + heyvacay-iac.

1. **Production subdomain.** In Route 53 (or your DNS host) create an **A / ALIAS** record for `ent.heyvacay.co` pointing at the Enterprise load balancer (ALB) created by heyvacay-iac. Add it to the ALB listener + ACM certificate (SAN) so HTTPS is valid.
2. **Internal Nova subdomain** (optional). Point `ent.nova.heyvacay.co` at the same service if you want the internal console on its own host; otherwise it stays at `/nova` on the app.
3. **Login screens** — already built at `/signin` (SSO buttons, work-email, and a domain-join hint) and `/signup`. No action beyond DNS; they render on the new domain automatically.
4. **Domain self-join backend.** The UI is done: on each company profile (Sales → Companies → a company → **Domain self-join**) you register the company's email domain and toggle auto-join. To make it enforce for real, the auth backend must, on email verification: match the verified domain to a company with self-join enabled and create the user at the **lowest (traveler)** role. Hand this spec to the backend team — the enterprise app reads the flag and surfaces the hint today.
5. **Verify.** Hit `https://ent.heyvacay.co/signin`, type a corporate-domain email, and confirm the "you'll be added automatically once verified" hint appears.

7 Everything already built for you DONE

Shipped on `claude/real-search` and covered by this UAT round — no action required.

- [ ] **Dark-mode & contrast fixes** — swept the whole app (incl. Nova) for invisible text; fixed the hotel/flights buttons and the white-on-white pricing calculator input.
- [ ] **Booking + booking-confirmed screens** rebuilt to match heyvacay.co exactly, including the confetti animation.
- [ ] **Stripe checkout** (Elements + saved card) routed through the same backend, same keys, same tracking tags.
- [ ] **Home-city prompt** for new users; attendees >50 mi away flagged as needing a hotel, and unbooked attendees light up **red** on the event dashboard.
- [ ] **Per-company commission tab** with an override checkbox; the commission dashboard lists which companies won't follow global changes.
- [ ] **C-Suite role** — only C-Suite manages the commission dashboard; reps can move company rates only 1% down / 2% up before it emails every C-Suite member and flags the CRM dashboard.
- [ ] **Nova booking management** — full cross-account console (cancel, special requests, detail drawer) matching heyvacay.co's admin, with the same information.
- [ ] **Events** — clearer city selection, click-to-autofill (city/dates), allow-list-only hotels with a traveler warning, and "booking for this event" context.
- [ ] **Allow-list policy bug fixed** — in-policy event bookings no longer force manager approval.
- [ ] **Natural-language reporting** — type a phrase (e.g. "amount for Lufthansa in 2018 by top 10 countries by report owner") and it builds the structured report.
- [ ] **Supplier names removed** from the trip-approval screen.
- [ ] **Access control** — employees no longer see admin controls; a scoped access-denied screen shows while keeping the sidebar.
- [ ] **Points consistency** — search and approval now award points the same way, and the **0.0 rating badge** is gone.
- [ ] **Expenses overhaul** — receipt scanning, virtual cards, per diems and calendar sync delivered.
- [ ] **Real HeyVacay logo** with the blue "Enterprise" lockup across the app, Nova, login, contracts and reports.
- [ ] **Interactive New Company flow** that captures account reps (points of contact) and domain self-join.

8 Quick reference — all env vars ACTION NEEDED

ENT_API_BASE_URL	Shared HeyVacay backend (flights, hotels, Stripe init). Point at the same backend as heyvacay.co.
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	`pk_live_…` — same key as heyvacay.co. Required for live checkout; the backend makes the charge.
STRIPE_SECRET_KEY (optional)	`sk_live_…` — only for the direct-to-Stripe fallback.
OPENTABLE_API_KEY or OPENTABLE_CLIENT_ID + OPENTABLE_CLIENT_SECRET	Dining. Optional `OPENTABLE_API_BASE`.
EMAIL_API_KEY / EMAIL_FROM	Transactional email + sender.
WEATHER_API_KEY	OpenWeather (active-trip view).
FLIGHT_STATUS_API_KEY / FLIGHT_STATUS_API_BASE	Live flight status (both required).

Golden rule: each secret goes into **AWS Secrets Manager** (us-west-2), is referenced in **heyvacay-iac → ent.tf** under the task's `secrets`, then a **redeploy** picks it up. Missing keys never break the app — they just keep that one feature in demo mode.